
EAGER-MODE DNN PROGRAM COMPILATION WITH MAGPY IN TVM

Ruihang Lai¹ Lauren Sands¹

ABSTRACT

PyTorch, the most popular machine learning framework, runs in an eager execution mode. Thus, unlike ML compilers, PyTorch programs cannot take advantage of optimizations that require knowledge of the entire ML model. TorchDynamo and TorchInductor attempted to address this issue by generating FX graphs and compiling them with JIT compilation; however, ML models with complex control flow are still not well supported by this approach. MagPy was created to address these issues and achieves a speedup over TorchDynamo and TorchInductor by taking advantage of the fact that data types and tensor shapes often remain the same across multiple runs of a function. However, MagPy is not currently compatible with the TVM compiler backend because MagPy cannot generate the graph-level TVM IR. We bridged this gap by implementing a graph translator to convert TorchFX graphs to the TVM IR. We found that our performance was on par with MagPy on other compiler backends, and improved over the baselines of eager-mode PyTorch and TorchDynamo. We hope that this will allow MagPy to be used in conjunction with other TVM optimizations in the future.

1 INTRODUCTION

Machine learning (ML) applications are now ubiquitous in everyday life. From enhancing image quality in photo editing apps to powering virtual assistants and enabling personalized recommendations, ML has transformed how we interact with technology. Underlying these advances are diverse categories of models tailored to specific tasks. In computer vision, convolutional neural networks (CNNs) such as ResNet (He et al., 2015) excel at recognizing and classifying visual patterns, forming the backbone of applications such as facial recognition and autonomous driving. Language models, especially the rapidly evolving class of large language models (LLMs) (Touvron et al., 2023; Rozière et al., 2024; Black et al., 2022; Vaswani et al., 2017; Xu et al., 2023), have revolutionized natural language understanding, enabling nuanced text generation, translation, and information retrieval. Meanwhile, diffusion models (Rombach et al., 2022; Podell et al., 2023) are redefining generative capabilities, producing high-quality synthetic images, videos, and more. These models illustrate the breadth and depth of the impact of machine learning across domains.

PyTorch (Paszke et al., 2019), as the most popular machine learning framework, adopts the eager execution mode to run models. The eager execution mode follows the execution order of the Python interpreter. For a tensor operation in the code, the eager execution mode only applies the tensor operation when the Python interpreter reaches this line.

Based on the eager execution mode, PyTorch offers great flexibility and convenience for users to customize and compose machine learning models, which contributes to the prosperity of the machine learning community. In this eager mode, machine learning frameworks fail to obtain a global picture of what the whole model looks like. As a result, eager-mode frameworks (such as PyTorch) usually miss the computational graph-level acceleration opportunities (e.g., operator fusion) of ML models and, therefore, can only ship suboptimal performance.

In contrast to eager execution mode, machine learning compilers (Chen et al., 2018; Lattner et al., 2021; Tillet et al., 2019) aim to first obtain entire ML models and optimize and accelerate models both globally across the entire computational graph and locally within single-tensor operators. These compilers typically provide program abstractions (or intermediate representations) of high-level computational graphs and low-level tensor operators, which are the foundation of the compiler optimizations. Though ML compilers can bring better performance when running models, describing ML models with these compilers' IRs often requires a significant amount of engineering and time. This inconvenience prevents people from deploying models with ML compilers.

Research has been conducted to address this challenge. Researchers have tried to bridge existing model implementations in PyTorch with ML compilers so that no manual modeling with ML compilers' IRs is needed. PyTorch later introduced its graph-level IR Torch FX (Reed et al., 2022), supporting the generation of an FX graph for a given ML

¹Carnegie Mellon University.

model by symbolic tracing. PyTorch2 (Ansel et al., 2024) further introduces TorchDynamo, which dynamically traces a given ML model with actual runtime input value and generates the traced FX graph. The traced FX graphs are further compiled by TorchInductor with just-in-time (JIT) compilation. However, the complexity and dynamism of Python bring extra difficulties for both symbolic and runtime tracing. For example, TorchDynamo has limited support for dynamic control flow, and as a result, ML models with complex control flows may not be well supported by the TorchDynamo approach. These difficulties usually lead to tracing incompleteness or even failure.

MagPy (Zhang et al., 2024) is a new method which overcomes some runtime tracing issues of TorchDynamo by monitoring the execution states of the Python interpreter when tracing. MagPy is designed based on three insights. First, most deep-learning programs have limited dynamics. Although programs are written in Python, usually the dynamism of data type, control flow, and runtime function dispatch remain the same across different batches. Second, only external values can affect program behavior, which makes it possible to significantly reduce the complexity of monitoring dynamics during graph instantiation, with the only need to validate the operations and generate guards to check whether external reads are unchanged to determine whether the graph is changed. Third, both the guard and the mock can be determined by analyzing the execution states of Python programs. The guard validates whether the input state of a new call matches the previous run, and the mock reproduces the final state of the previous run. Both depend only on the runtime states instead of the logic of the ML model. Based on these insights, MagPy proposes RefGraph (Reference Graph) to record program states during program execution, and thus effectively tracks ML models. MagPy constructs the Torch FX graph on the fly when tracing ML models, and leverages TorchInductor or other machine learning compilers to optimize the models with JIT compilation. MagPy claims an average runtime performance improvement of up to 2.88× compared to existing graph instantiation techniques when using the same deep learning compiler to compile operator graphs.

We realize that the original MagPy may miss some other optimization opportunities due to its backend ML compiler choices. TVM (Chen et al., 2018) is a machine learning compiler that features state-of-the-art performance and optimization customizability. However, TVM is not yet a MagPy compiler backend, because MagPy cannot generate the graph-level IR (Lai et al., 2023) in TVM. To leverage TVM for potential better performance, our project aims to support MagPy with the TVM backend and validate the performance improvement over MagPy and its baselines.

There are multiple options to achieve this goal. One could

directly generate the TVM IR on the fly while tracing ML models in MagPy. Unfortunately, we found that the MagPy implementation is closely tied to the torch FX graph, which makes it challenging to change to TVM IR generation. Therefore, we choose to reuse the torch FX graphs generated by MagPy and implement a graph IR translator to convert torch FX graphs to the TVM IR. With this approach, we can bypass the complications of MagPy and torch FX graphs. In addition to the graph IR translator, we implement other necessary runtime modules to bridge the MagPy tracing with the TVM backend. Evaluation results show that MagPy with the TVM backend can achieve performance on par with that of MagPy on other backends, which aligns with our expectation that MagPy improves over the basic PyTorch or TorchDynamo baselines.

We summarize the technical contribution of this project as follows:

- We bridge the ML model tracing in MagPy and TVM, one of the state-of-the-art ML compilers, by implementing a graph translator that converts torch FX graphs to TVM IR. The evaluation shows that this integration achieves performance on par with MagPy in other ML compiler backends.

2 OUR APPROACH

2.1 System Overview

This section presents the overview of this project. As shown in Figure 1, MagPy with the TVM backend makes up the entire just-in-time compilation system. We leverage MagPy to run just-in-time (JIT) compilation for ML models and generate torch FX graph representations of the model. Then we translate the torch FX graph to TVM’s graph-level IR, and finish the model compilation with TVM’s compilation pipeline. We also include necessary runtime support to convert PyTorch tensor inputs to the TVM tensor data structure, run the model through the TVM runtime, and convert the output tensors back to PyTorch tensors.

2.2 JIT Compilation in MagPy

In this section, we introduce the JIT compilation foundation of this project. JIT compilation is a technique in which the model is compiled during runtime rather than prior to execution. It combines features of both interpretation and ahead-of-time (AOT) compilation. A JIT compiler usually adopts a program cache. For every model that needs compilation, the system looks in the cache to see if the same program has already been compiled. Once compiled, the compiled model is cached and reused, which improves performance compared to interpreting the same code repeatedly.

Given that many variables in DNN execution remain con-

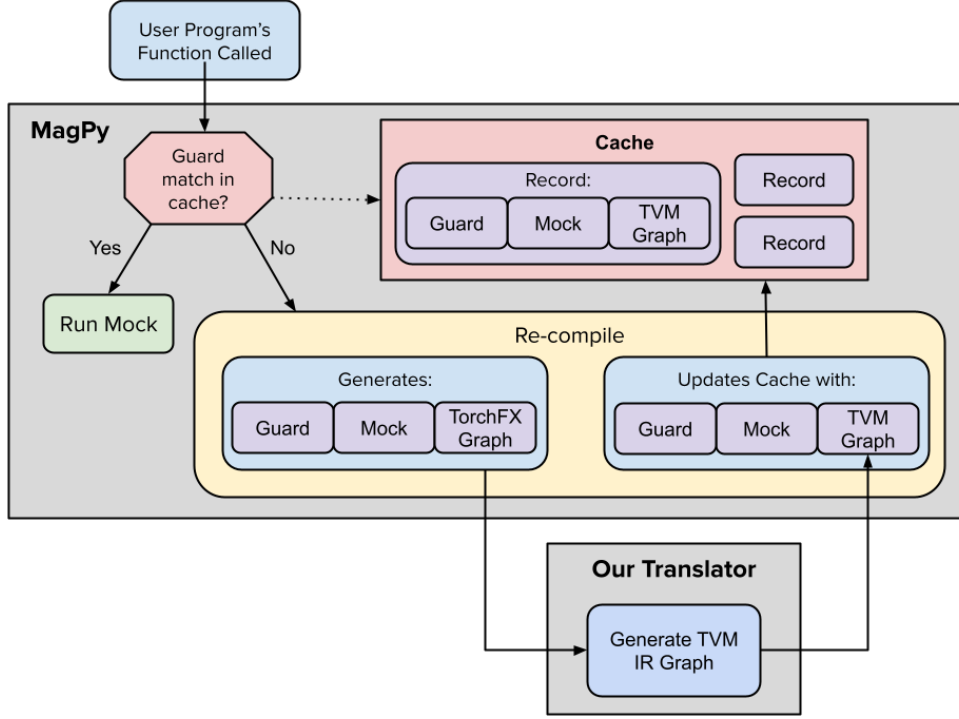


Figure 1. Workflow of guard-based just-in-time compilation with our graph translator.

stant across different batches, ML models can be specialized with these unchanged variables by JIT compilation. This is to say, in the most common cases, only a few variables (such as batch size and data type) changes across different runs. Therefore, we can effectively cache all the compiled models for each individual set of variables.

To ensure correctness and safety, it is necessary to check whether two sets of variables are the same or different across runs. MagPy introduces the *guard functions* to address this problem. During JIT compilation, the JIT compiler collects all the changeable variables. These variables may include function inputs, global variables, any external states, etc. Whenever a set of concrete values is specified for these variables, the output of the execution of the ML model will be uniquely determined. Therefore, for each run, the JIT compiler generates a guard function that takes in a set of variables, and returns whether this set of variables matches the values of changeable variables in this run (i.e., check shapes and data types). When a new program needs compilation, MagPy first checks if any guard function of cached programs can be passed. If there is a guard match, the cached version will be used and no re-compilation takes place. Otherwise, MagPy runs the JIT compilation for the program, generates the corresponding guard function on the fly, and caches the compiled program and the guard function for possible future reuse.

2.3 Torch FX Graph Components

The torch FX graph is an intermediate stage during program compilation in this project. When compiling a program, we first generate the torch FX graph, then translate the torch FX graph to TVM’s graph-level IR for further compilation. We introduce the torch FX graph in this section.

Torch FX is a framework within PyTorch designed to capture and transform the computational graphs of PyTorch programs. At the core of Torch FX is the Graph structure, which represents the computation in a PyTorch program as a sequence of operations. Each operation in the graph is described as a Node, with distinct types of nodes capturing different kinds of computational steps.

Below are the primary node types of Torch FX graphs that represent function calls. Based on the properties (e.g., stateful or stateless) of target functions being called, function calls are classified into three types: `call_module`, `call_function` and `call_method`.

call_module A `call_module` node represents the invocation of a submodule within a PyTorch `nn.Module`. The `call_module` node is used when the computation involves calling a module, such as layers (e.g., `nn.Linear`) or custom modules defined by the user. Each `call_module` node is associated with the name of the

submodule being invoked, arguments passed to the submodule, and a reference to the submodule in the parent module’s hierarchy.

call_function A `call_function` node represents the invocation of a free function or a built-in function, such as those in the PyTorch functional API or standard Python functions. This node type encapsulates stateless functional transformations (e.g., `torch.add`), where no attributes of the `nn.Module` are involved.

call_method A `call_method` node represents the invocation of a method on a Python object. This is commonly used for tensor-specific operations or object-oriented computations. Typical `call_method` examples include tensor methods such as `.view` for creating a tensor view in a new shape, `.sum` for accumulation along an axis, or `.to` for data type conversion.

In addition to these nodes for function calling, torch FX graphs contain other nodes to capture model inputs, outputs, and other attributes. A `placeholder` node represents an input to the computational graph. These are the entry points for data and parameters passed into the module. An `output` node denotes the output of the graph. This is the terminal point of the computation, capturing what the graph produces. An `get_attr` node refers to an attribute of the parent module, such as parameters or buffers (e.g., weights or running statistics).

Torch FX graphs capture the sequence of these nodes, forming a directed acyclic graph (DAG). The edges represent data dependencies between nodes. Each node can be inspected, modified, or reordered to achieve transformations like optimization, quantization, or export. This structured representation is what enables powerful programmatic manipulation of PyTorch models while preserving their functionality and modularity.

2.4 Graph Translator

In this section, we introduce the graph translator that translates torch FX graphs to the graph-level IR in TVM.

Given that the model tracing in MagPy always starts with a given set of concrete values for all the changeable variables, the code path is always uniquely determined. Therefore, the torch FX graphs constructed on the fly are linear in structure and free of complicated control flow. This property significantly reduces the efforts needed for translation, as the translator can essentially be implemented as a linear scan over all the nodes in the torch FX graph in topological order.

At the core of the graph translator is a conversion registry, where we register one entry for each native `nn.Module` or

Algorithm 1 Graph translation algorithm.

```

1: Input: A torch FX Graph  $g$  to be translated.
2: Output: The TVM graph-level IR  $f$  after translation.
3: Create a fresh variable for every placeholder node in  $g$ . Store these variables
   in list func_params, which is the parameter list of the translated function.
4:  $f \leftarrow$  a new empty TVM graph-level function with parameters func_params
5: env_table  $\leftarrow$  an empty mapping from torch FX nodes to TVM IR variables
6: for all FX graph node node in  $g$  in topological order do
7:   if node.op is placeholder then
8:     Skip as it has already been processed.
9:   else if node.op is call_module then
10:    module_cls  $\leftarrow$  the type of the callee module node.target
11:    conversion_func  $\leftarrow$  the conversion function for module_cls in
      the conversion registry
12:    env_table[node]  $\leftarrow$  conversion_func(node.args, f)
      (invoke the conversion function, and construct  $f$  on the fly)
13:   else if node.op is call_function then
14:    conversion_func  $\leftarrow$  the conversion function for the callee function
      node.target
15:    env_table[node]  $\leftarrow$  conversion_func(node.args, f)
      (invoke the conversion function, and construct  $f$  on the fly)
16:   else if node.op is call_method then
17:    conversion_func  $\leftarrow$  the conversion function for the callee method
      node.target
18:    env_table[node]  $\leftarrow$  conversion_func(node.args, f)
      (invoke the conversion function, and construct  $f$  on the fly)
19:   else if node.op is get_attr then
20:    env_table[node]  $\leftarrow$  the target attribute node.target of the mod-
      ule of node.
21:   else if node.op is output then
22:     Collect all the output variables in node.
23:   Wrap up and finish the construction of  $f$  with the output variables.
24:   end if
25: end for

```

function in PyTorch. The entry contains the corresponding logic that constructs equivalent tensor operations in TVM. We use Python functions to represent this logic, which provides the programming flexibility for us to support both simple and complex translation cases of `nn.Module` and functions. For simple cases (e.g., `torch.add`, `torch.exp`), the registered function can be as simple as creating one TVM operator call and adding it into the graph IR being constructed. Complex cases require more careful handling in the registered function to properly cover all possible semantics of the torch FX graph node. Tensor indexing/slicing (or formally `__getitem__` in Python) is an example of such a complex case. In PyTorch, `__getitem__` has multiple meanings. `a[0, 0]` takes the concrete value at a specific position of a tensor. `a[i:j:k, 0]` takes a slice of a tensor, where the indices i , j and k can even be negative under the Python negative indexing rule. `a[idx]` takes the values from a tensor with regard to another indexing tensor. To correctly handle such complexity, we apply argument pattern matching on the “indices”, or the arguments of the `__getitem__` function, to dispatch different patterns to different graph operations in TVM. A conversion function takes in the arguments of the function call being converted, and returns the constructed TVM IR variable corresponding to the torch FX graph node of the current function call.

Algorithm 1 shows the pseudocode of this graph translation process. We start with an empty TVM graph-level function, which will be constructed on the fly when we iterate over all

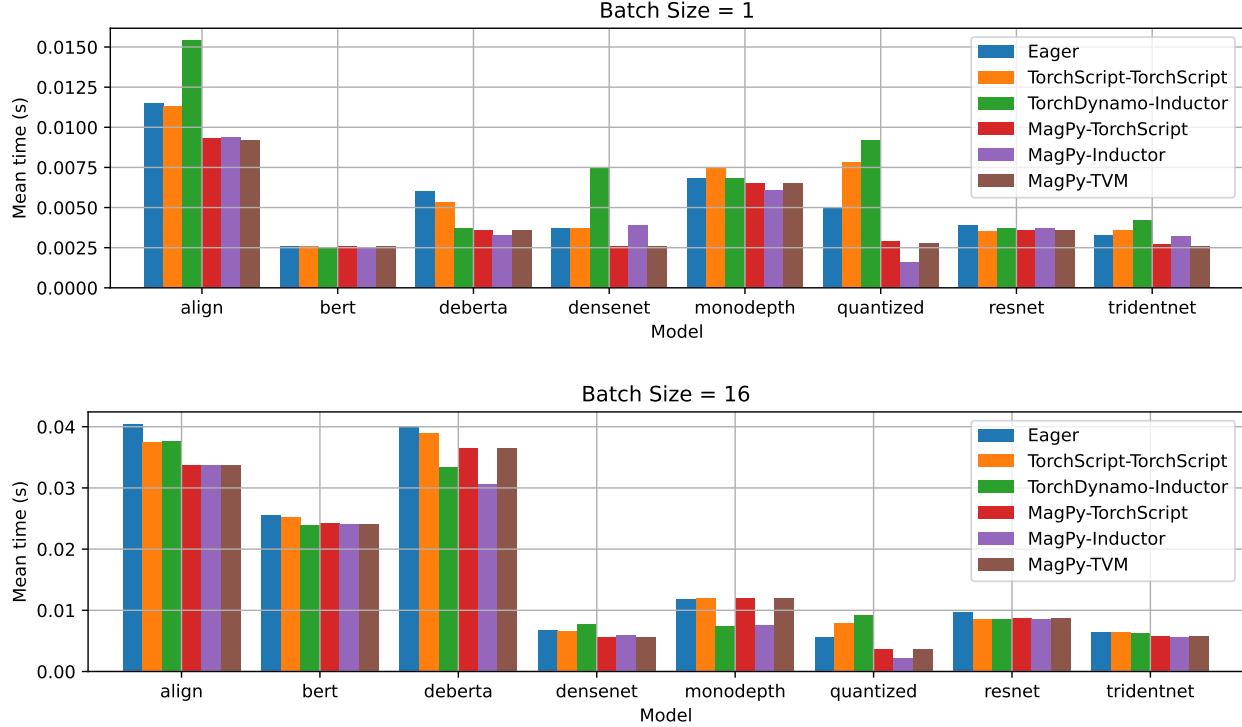


Figure 2. End-to-end inference on NVIDIA RTX 4090. xxx-yyy means tracing the model via method xxx and compiling the model with method yyy.

nodes in the torch FX graph. The first step is to collect the function parameters by creating a fresh variable for every placeholder node in the FX graph. After that, we go through all nodes in the FX graph in the topological order. For each node, we look up its corresponding conversion function in the registry based on its node type (whether it is `call_module`, `call_function` or other types). We then invoke the conversion function, passing in `f` and the arguments of the node. The conversion function creates new operations and adds them to `f`, and returns the TVM IR variable of the last operation. This variable will be saved in an environment table for further argument lookup when converting other FX graph nodes.

3 EVALUATION

3.1 Evaluation Setup

Platform. The evaluation is performed on one NVIDIA RTX 4090 with AMD Ryzen 9 7950X CPU. The software versions are Python 3.9, CUDA 11.8, and GCC 11.4.

Baselines. Our baselines include two parts. Firstly, we compare our system with graph instantiation techniques, including analysis-based frameworks TorchDynamo (Ansel et al., 2024) and TorchScript (in PyTorch v2.0.1). Secondly,

we include MagPy as the baseline as well.

Benchmark. We adopt the official artifact evaluation benchmark of MagPy¹ and run the first benchmark (for Figure 15 in MagPy paper). Specifically, the benchmark measure the execution time of 8 representative DNN models. These models cover typical DNN architectures like CNN and transformers and include popular scenarios like multi-model and quantization. These models contain both classical models like BERT (Devlin et al., 2019) and ResNet (He et al., 2015) that have been manually simplified to accommodate the currently limited graph instantiation ability, and models that are a bit less popular so not yet been simplified, including ALIGN (Jia et al., 2021), DeBERTa (He et al., 2021), DenseNet (Huang et al., 2018), MonoDepth (Godard et al., 2017), TridentNet (Li et al., 2019). We also include a quantized ResNet model with the quantization algorithm from (Banner et al., 2018). We include instructions in the submitted source code for result reproduction.

3.2 Results

We evaluated two representative batch sizes of 1 and 16 respectively, and Figure 2 shows our benchmark results. In

¹<https://github.com/heheda12345/MagPy-AE>

both batch sizes, our integration of MagPy into the TVM backend (denoted as “MagPy-TVM”) outperforms the baselines without MagPy model tracing. This result shows the effectiveness of MagPy-style model tracing, which enables more computational graph-level optimization opportunities by obtaining larger computational graphs and reducing the tracing failure, which aligns with our expectations.

In addition, MagPy-TVM has aligned performance with other MagPy baselines. It achieves up to 3% of execution time reduction over other MagPy baselines. Notably, our method MagPy-TVM has similar results to the MagPy-TorchScript method. These two methods outperform the MagPy-Inductor baseline in some cases, while being slightly worse in some other cases. We identify that one possible reason that may cause slowdown in MagPy-TVM is that we did not yet apply auto-tuning for loop-level tensor programs in TVM, and instead only applies of default tensor program optimization rules. These optimization rules may be suboptimal for complicated operators such as convolutions. We anticipate more performance improvement with more careful optimization on low-level tensor programs in the models.

4 SURPRISES AND LESSONS LEARNED

Here are the lessons we learned when working on this project.

Python version dependence. MagPy model tracing relies on monitoring the execution states of the Python interpreter. However, the Python execution state data structures in Python interpreter changes over Python versions, which means that the implementation of this monitoring are Python version-dependent, and it is not possible to implement the monitoring that is generally applicable to all Python versions. Due to this reason, the current MagPy implementation only works for Python 3.9, and it requires much extra engineering efforts to support other Python versions. This significantly hinders a broader application of the MagPy model tracing.

Torch FX graph dependence. Our initial plan for this project was to reimplement MagPy to directly generate the graph-level IR Relax in TVM. However, after tracing the system code, we realized that the MagPy implementation is highly coupled with the FX graph data structure. We estimated that there will be huge engineering efforts needed if we are going to construct TVM graph-level IR directly from the model tracer in MagPy. As a reconciliation, we chose to make torch FX graph an intermediate stage of the compilation. Though not being able to directly generate TVM graph-level IR, this approach offers the flexibility of integrating MagPy with other ML compilers.

5 CONCLUSIONS AND FUTURE WORK

We introduce eager-mode DNN program compilation with MagPy for just-in-time compilation in TVM. The core of this project is a flexible translator from torch FX graph to the graph-level IR in TVM, which allows for leveraging TVM for DNN model compilation. The integration has aligned performance with the MagPy baseline with other backend compilers. Future work of this project can be improving the performance with more careful loop-level tensor program optimizations.

REFERENCES

- Ansel, J., Yang, E., He, H., Gimershein, N., Jain, A., Voznesensky, M., Bao, B., Bell, P., Berard, D., Burovski, E., Chauhan, G., Chourdia, A., Constable, W., Desmaison, A., DeVito, Z., Ellison, E., Feng, W., Gong, J., Gschwind, M., Hirsh, B., Huang, S., Kalambarkar, K., Kirsch, L., Lazos, M., Lezcano, M., Liang, Y., Liang, J., Lu, Y., Luk, C. K., Maher, B., Pan, Y., Puhersch, C., Reso, M., Saroufim, M., Siraichi, M. Y., Suk, H., Zhang, S., Suo, M., Tillet, P., Zhao, X., Wang, E., Zhou, K., Zou, R., Wang, X., Mathews, A., Wen, W., Chanan, G., Wu, P., and Chintala, S. Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS ’24, pp. 929–947, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400703850. doi: 10.1145/3620665.3640366. URL <https://doi.org/10.1145/3620665.3640366>.
- Banner, R., Hubara, I., Hoffer, E., and Soudry, D. Scalable methods for 8-bit training of neural networks, 2018. URL <https://arxiv.org/abs/1805.11046>.
- Black, S., Biderman, S., Hallahan, E., Anthony, Q., Gao, L., Golding, L., He, H., Leahy, C., McDonell, K., Phang, J., Pieler, M., Prashanth, U. S., Purohit, S., Reynolds, L., Tow, J., Wang, B., and Weinbach, S. Gpt-neox-20b: An open-source autoregressive language model, 2022.
- Chen, T., Moreau, T., Jiang, Z., Zheng, L., Yan, E., Shen, H., Cowan, M., Wang, L., Hu, Y., Ceze, L., et al. Tvm: An automated end-to-end optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pp. 578–594, 2018.
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019. URL <https://arxiv.org/abs/1810.04805>.

- Godard, C., Aodha, O. M., and Brostow, G. J. Unsupervised monocular depth estimation with left-right consistency, 2017. URL <https://arxiv.org/abs/1609.03677>.
- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition, 2015. URL <https://arxiv.org/abs/1512.03385>.
- He, P., Liu, X., Gao, J., and Chen, W. DeBERTa: Decoding-enhanced bert with disentangled attention, 2021. URL <https://arxiv.org/abs/2006.03654>.
- Huang, G., Liu, Z., van der Maaten, L., and Weinberger, K. Q. Densely connected convolutional networks, 2018. URL <https://arxiv.org/abs/1608.06993>.
- Jia, C., Yang, Y., Xia, Y., Chen, Y.-T., Parekh, Z., Pham, H., Le, Q. V., Sung, Y., Li, Z., and Duerig, T. Scaling up visual and vision-language representation learning with noisy text supervision, 2021. URL <https://arxiv.org/abs/2102.05918>.
- Lai, R., Shao, J., Feng, S., Lyubomirsky, S. S., Hou, B., Lin, W., Ye, Z., Jin, H., Jin, Y., Liu, J., Jin, L., Cai, Y., Jiang, Z., Wu, Y., Park, S., Srivastava, P., Roesch, J. G., Mowry, T. C., and Chen, T. Relax: Composable abstractions for end-to-end dynamic machine learning, 2023. URL <https://arxiv.org/abs/2311.02103>.
- Lattner, C., Amini, M., Bondhugula, U., Cohen, A., Davis, A., Pienaar, J., Riddle, R., Shpeisman, T., Vasilache, N., and Zinenko, O. Mlir: Scaling compiler infrastructure for domain specific computation. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pp. 2–14. IEEE, 2021.
- Li, Y., Chen, Y., Wang, N., and Zhang, Z. Scale-aware trident networks for object detection, 2019. URL <https://arxiv.org/abs/1901.01892>.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
- Podell, D., English, Z., Lacey, K., Blattmann, A., Dockhorn, T., Müller, J., Penna, J., and Rombach, R. Sdxl: Improving latent diffusion models for high-resolution image synthesis, 2023. URL <https://arxiv.org/abs/2307.01952>.
- Reed, J., DeVito, Z., He, H., Ussery, A., and Ansel, J. torch.fx: Practical program capture and transformation for deep learning in python. In Marculescu, D., Chi, Y., and Wu, C. (eds.), *Proceedings of Machine Learning and Systems*, volume 4, pp. 638–651, 2022. URL https://proceedings.mlsys.org/paper_files/paper/2022/file/7c98f9c7ab2df90911da23f9ce72ed6e-Paper.pdf.
- Rombach, R., Blattmann, A., Lorenz, D., Esser, P., and Ommer, B. High-resolution image synthesis with latent diffusion models, 2022. URL <https://arxiv.org/abs/2112.10752>.
- Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., Adi, Y., Liu, J., Sauvestre, R., Remez, T., Rapin, J., Kozhevnikov, A., Evtimov, I., Bitton, J., Bhatt, M., Ferrer, C. C., Grattafiori, A., Xiong, W., Défossez, A., Copet, J., Azhar, F., Touvron, H., Martin, L., Usunier, N., Scialom, T., and Synnaeve, G. Code llama: Open foundation models for code, 2024.
- Tillet, P., Kung, H.-T., and Cox, D. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pp. 10–19, 2019.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., Bikel, D., Blecher, L., Ferrer, C. C., Chen, M., Cucurull, G., Esiobu, D., Fernandes, J., Fu, J., Fu, W., Fuller, B., Gao, C., Goswami, V., Goyal, N., Hartshorn, A., Hosseini, S., Hou, R., Inan, H., Kardas, M., Kerkez, V., Khabsa, M., Kloumann, I., Korenev, A., Koura, P. S., Lachaux, M.-A., Lavril, T., Lee, J., Liskovich, D., Lu, Y., Mao, Y., Martinet, X., Mihaylov, T., Mishra, P., Molybog, I., Nie, Y., Poulton, A., Reizenstein, J., Rungta, R., Saladi, K., Schelten, A., Silva, R., Smith, E. M., Subramanian, R., Tan, X. E., Tang, B., Taylor, R., Williams, A., Kuan, J. X., Xu, P., Yan, Z., Zarov, I., Zhang, Y., Fan, A., Kambadur, M., Narang, S., Rodriguez, A., Stojnic, R., Edunov, S., and Scialom, T. Llama 2: Open foundation and fine-tuned chat models, 2023.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L. u., and Polosukhin, I. Attention is all you need. In Guyon, I., Luxburg, U. V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
- Xu, C., Sun, Q., Zheng, K., Geng, X., Zhao, P., Feng, J., Tao, C., and Jiang, D. Wizardlm: Empowering large language models to follow complex instructions. *arXiv preprint arXiv:2304.12244*, 2023.

Zhang, C., Dong, R., Wang, H., Zhong, R., Chen, J., and Zhai, J. MAGPY: Compiling eager mode DNN programs by monitoring execution states. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pp. 683–698, Santa Clara, CA, July 2024. USENIX Association. ISBN 978-1-939133-41-0. URL <https://www.usenix.org/conference/atc24/presentation/zhang-chen>.